

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Реализация эталонного теста для
сравнительного анализа
производительности колоночной
аналитической СУБД с поддержкой
позиционного представления

Щека Дмитрий Вадимович

Отчёт по преддипломной практике
в форме «Эксперимент»

Научный руководитель:
ассистент кафедры информационно-аналитических систем Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор популярных эталонных тестов	6
2.1. ClickBench	7
2.2. TPC-H	8
2.3. TPC-DS	9
2.4. SSB	10
2.5. JOB	11
2.6. Выводы	13
3. Составление физических планов запросов	14
3.1. Основные принципы	14
3.2. Интересные случаи из составленных планов	15
3.3. Результаты	18
4. Реализация	19
5. Эксперименты	21
5.1. Условия эксперимента	21
5.2. Результаты	22
5.3. Интерпретация результатов	23
Заключение	31
Список литературы	32

Введение

СУБД развиваются длительный срок, и за это время появилось множество различных техник, оптимизаций и подходов. С увеличением количества данных возрастает необходимость обрабатывать их в разумные сроки, поэтому одним из основных направлений развития СУБД является повышение производительности. Из этого возникает потребность не только измерять время выполнения отдельных запросов, но и сравнивать разные системы на одинаковой нагрузке. Для этого используются эталонные тесты, которые содержат наборы SQL-запросов и данных. Хороший эталонный тест должен отражать характерные способы использования СУБД и проверять разные аспекты системы: сканирование данных, фильтрацию, соединения, агрегации, сортировки и работу с подзапросами.

Однако итоговое время выполнения запроса само по себе не всегда объясняет, почему одна система оказывается быстрее другой. На производительность сильно влияет физический план: порядок чтения таблиц, выбор алгоритмов соединения, момент применения фильтров, способ агрегации и материализации промежуточных результатов. Поэтому при сравнении СУБД важно рассматривать не только численные результаты, но и то, какие физические планы привели к этим результатам. Такой анализ позволяет понять, какие свойства движка дают преимущество в конкретном запросе, а какие отсутствующие оптимизации ограничивают производительность.

PosDB [8] — это аналитическая колоночная СУБД [1], которая использует механизм позиционного представления данных (Position-Enabled Column-Store). На данный момент в ней отсутствует оптимизатор, а из эталонных тестов реализован только SSB [9]. Этот тест полезен для первичной проверки колоночного движка, но его запросы сравнительно просты и не покрывают ряд случаев, важных для дальнейшего развития системы: подзапросы, большое число соединений, разные варианты сортировки и более сложные условия фильтрации. Поэтому для развития PosDB требуется выбрать более сложный эталонный тест и на

его основе определить, какие возможности движка требуют доработки.

В данной работе будет выбран новый эталонный тест для PosDB и составлены физические планы его запросов. Поскольку в PosDB сейчас отсутствует оптимизатор, планы будут составляться вручную с учетом известных принципов оптимизации аналитических запросов. После этого будет проведено сравнение PosDB с PostgreSQL. Для PostgreSQL будут рассматриваться планы, полученные с помощью EXPLAIN, а для PosDB — вручную составленные планы. Такое сравнение позволит не только определить, какая система быстрее на конкретных запросах, но и объяснить причины различий: за счет каких операторов, порядка соединений или способов обработки данных достигается преимущество одной из систем. Отдельное внимание будет уделено тому, каких механизмов не хватает PosDB для того, чтобы стабильно превосходить PostgreSQL на выбранной аналитической нагрузке.

1. Постановка задачи

Целью работы является исследование производительности движка PosDB на запросах из эталонного теста для аналитических СУБД и сравнение полученных результатов с PostgreSQL. При этом сравнение не ограничивается измерением времени выполнения: для каждого запроса требуется сопоставить физический план PosDB с планом, построенным PostgreSQL, и объяснить причины различий в производительности.

Такой подход позволяет оценить не только текущее состояние PosDB, но и направления ее дальнейшего развития. Если PosDB оказывается быстрее, это показывает, какие свойства колоночного движка и позиционного представления дают преимущество на данной нагрузке. Если быстрее оказывается PostgreSQL, анализ плана позволяет понять, каких операторов, оптимизаций или механизмов планирования не хватает PosDB для улучшения результата.

Для достижения цели были поставлены следующие задачи:

1. Выбрать эталонный тест, подходящий для сравнения аналитических СУБД и содержащий более сложные запросы, чем уже реализованный в PosDB тест.
2. Реализовать внутри PosDB недостающую функциональность, необходимую для выполнения запросов из выбранного эталонного теста.
3. Составить эффективные физические планы для запросов выбранного эталонного теста с учетом особенностей PosDB.
4. Замерить выполнение эталонного теста и сравнить результаты выполнения запросов, сопоставив их с физическими планами обеих систем.
5. На основе анализа результатов определить, какие особенности PosDB дают преимущество, а какие отсутствующие операторы или оптимизации ограничивают производительность системы.

2. Обзор популярных эталонных тестов

За время развития СУБД было разработано множество различных эталонных тестов. В этой секции рассмотрим несколько таких тестов и выберем один из них для дальнейшей реализации в PosDB. Поскольку в этой работе нас интересует не только сравнение времени выполнения, но и поиск перспективных направлений для развития движка, эталонный тест должен проверять те случаи, которые сейчас слабо покрыты уже реализованными тестами.

Выбор эталонного теста в данной работе не сводится к выбору наиболее популярного или наиболее сложного теста. Требуется найти компромисс между покрытием важных для PosDB механизмов и затратами на реализацию. Поэтому далее тесты сравниваются по следующим характеристикам:

1. **Воспроизводимость и признанность.** Учитывается наличие открытой спецификации, генератора данных, набора запросов и опубликованных результатов. Популярность в академической среде можно дополнительно оценивать числом упоминаний в ACM Digital Library, однако такую метрику нужно фиксировать вместе со строкой поиска и датой, потому что результат зависит от формулировки запроса.
2. **Покрываемость механизмов СУБД.** Для PosDB важны не только простые сканирования и агрегации, но и соединения нескольких таблиц, подзапросы, сортировки, ограничения числа строк, операции с датами и строками. Поэтому отдельно учитывается число таблиц, число запросов и наличие нетривиальных SQL-конструкций.
3. **Свойства данных.** Синтетические данные удобны для масштабирования, но часто имеют более простые распределения. Реальные данные лучше выявляют проблемы статистики и оценки кардинальностей, но обычно хуже масштабируются. В нашем случае сравнивается только движок, а вычислительные ресурсы ограни-

чены, из-за чего синтетический тест становится предпочтительнее.

4. **Затраты на внедрение.** Чем больше таблиц, запросов и синтаксических конструкций используется в тесте, тем больше требуется доработок в PosDB. В рамках данной работы слишком большой объем предварительной реализации может сместить фокус с анализа физических планов на поддержку SQL-синтаксиса.
5. **Полезность для дальнейших оптимизаций.** Тест должен не только дать итоговые числа производительности, но и показать, какие оптимизации и операторы стоит реализовывать дальше.

2.1. ClickBench

ClickBench¹ был создан разработчиками ClickHouse для сравнения аналитических СУБД. Он моделирует нагрузку, характерную для веб-аналитики и мониторинга. Набор данных получен из реального трафика одной из крупных платформ веб-аналитики и анонимизирован с сохранением основных распределений. В стандартном варианте используется одна широкая таблица примерно на 10^8 строк и 43 запроса.

Достоинства:

- Реальные распределения данных позволяют проверить эффективность сканирования, фильтрации, сжатия и работы с широкими таблицами.
- Тест легко воспроизводится, а на официальном сайте поддерживается публичная таблица результатов для большого числа систем.
- Одна таблица и небольшое число запросов делает внедрение сравнительно простым.

Недостатки:

¹<https://benchmark.clickhouse.com/>, дата обращения: 04.03.2026.

- Все запросы выполняются над одной плоской таблицей, поэтому тест не проверяет соединения и выбор порядка соединений.
- Такая модель данных отличается от классических хранилищ со стандартными схемами данных, поэтому результаты сильнее отражают эффективность сканирования одной широкой таблицы, чем работу СУБД в целом.

2.2. TPC-H

TPC-H [2] — это стандартный тест TPC для систем поддержки принятия решений. Он состоит из 22 бизнес-ориентированных SQL-запросов над 8 таблицами. Данные создаются генератором и масштабируются с помощью коэффициента масштабирования, поэтому один и тот же набор запросов можно запускать на разных объемах данных.

Достоинства:

- Тест хорошо известен и часто используется как в индустрии, так и в академических работах.
- Набор запросов включает соединения нескольких таблиц, агрегации, сортировки, вложенные подзапросы и разнообразные предикаты. Это делает его существенно полезнее для PosDB, чем тесты только на сканирование одной таблицы.
- По сравнению с TPC-DS он имеет умеренный размер: 22 запроса и 8 таблиц. Это снижает стоимость реализации, но сохраняет достаточно сложные случаи для анализа физических планов.
- Наличие формальной спецификации и генератора данных упрощает воспроизведение результатов.

Недостатки:

- Данные являются синтетическими и имеют более регулярные распределения, чем реальные пользовательские данные. Поэтому

TPC-H не всегда хорошо выявляет проблемы оценки кардинальностей. Однако в нашем случае из-за необходимости сравнения именно движка, этот недостаток не является критичным.

- Модель данных отражает более классический вариант хранилища данных и не покрывает некоторые современные паттерны, например оконные функции, сложные WITH-запросы и работу с полуструктурированными данными.
- Из-за известности теста многие системы хорошо оптимизированы именно под TPC-H, поэтому результаты нужно интерпретировать аккуратно.

2.3. TPC-DS

TPC-DS [7] можно рассматривать как более современный и широкий тест для систем поддержки принятия решений. Он моделирует работу крупного розничного продавца с несколькими каналами продаж: магазины, каталоги и интернет. Схема данных имеет вид “снежинки” и включает несколько таблиц фактов и таблиц измерений. Набор запросов состоит из 99 шаблонов, которые с помощью генератора превращаются в исполняемые SQL-запросы.

Достоинства:

- TPC-DS покрывает значительно более широкий набор аналитических сценариев, чем TPC-H: отчетные запросы, произвольные аналитические запросы (ad-hoc запросы), итеративные OLAP-запросы и запросы для интеллектуального анализа данных (Data Mining).
- Наличие нескольких таблиц фактов и большого числа измерений лучше соответствует современным хранилищам данных.
- Запросы используют более разнообразный SQL: вложенные подзапросы, сложные агрегации, WITH, оконные функции и другие конструкции SQL-99 с OLAP-расширениями.

Недостатки:

- Полная реализация требует поддержки большого числа таблиц, 99 запросов и существенно более широкого подмножества SQL. Для текущего состояния PosDB это делает стоимость внедрения слишком высокой.
- Запросы часто требуют адаптации под SQL-диалект конкретной СУБД. Такие изменения допускаются спецификацией как небольшие модификации запросов, но это увеличивает объем ручной работы.
- Из-за большой сложности TPC-DS может оказаться менее удобным первым шагом: значительная часть времени уйдет на поддержку синтаксиса, а не на анализ физических планов и оптимизаций.

2.4. SSB

Star Schema Benchmark (SSB) [6] был создан на основе TPC-H, но переработан под схему “звезда”. В отличие от TPC-H, таблицы `orders` и `lineitem` объединены в одну таблицу фактов `lineorder`, а вокруг нее расположены таблицы измерений `customer`, `part`, `supplier` и `date`. Набор включает 13 запросов, объединенных в несколько групп, которые моделируют реальную работу аналитика.

Достоинства:

- Схема “звезда” хорошо подходит для аналитических колоночных СУБД и позволяет проверять типичные операции над таблицей фактов и таблицами измерений.
- Тест прост в понимании и реализации, поэтому удобен для первичной проверки движка.
- Небольшой набор запросов позволяет быстро получить базовое сравнение производительности.

Недостатки:

- Запросы устроены существенно проще, чем в TPC-H и TPC-DS: в них нет подзапросов, самосоединений и сложных комбинаций операторов.
- Этот тест уже реализован в PosDB, поэтому его дальнейшее использование не дает достаточно нового материала для поиска слабых мест системы.

2.5. JOB

Join Order Benchmark (JOB) [4] был предложен для проверки того, насколько хорошо оптимизатор выбирает порядок соединения таблиц. В отличие от TPC-H и TPC-DS, JOB использует реальные данные IMDb. Эти данные содержат корреляции и неравномерные распределения, что делает оценку кардинальностей сложнее, чем на синтетических данных.

Тест включает 113 запросов над 21 таблицей. Запросы содержат большое количество соединений и специально сфокусированы на проблеме выбора порядка соединений. При этом их структура намеренно ограничена: это в основном один блок выборки, проекции и соединений с фильтрами по базовым таблицам.

Достоинства:

- JOB хорошо выявляет проблемы оценки кардинальностей и выбора порядка соединений.
- Реальные данные IMDb делают тест полезным для исследования оптимизаторов, потому что в них есть естественные корреляции между значениями.
- Большое число запросов с множественными соединениями позволяет детально исследовать один важный класс проблем оптимизации.

Недостатки:

- JOB не является полноценной заменой TPC-H или TPC-DS: в нем нет широкого набора аналитических операций, таких как GROUP BY, внешние соединения, подзапросы и сложные выражения.
- Исходный набор данных имеет размер всего несколько гигабайт и плохо подходит для проверки масштабируемости на больших объемах.
- Основной фокус JOB — работа оптимизатора. Поскольку в PosDB на данный момент физические планы составляются вручную, этот тест менее полезен как первый выбор для сравнения движка в целом.

Таким образом, выбор эталонного теста в данной работе не сводится к одному признаку вроде популярности или числа запросов. Для PosDB важнее, чтобы тест одновременно содержал новые нетривиальные случаи, оставался воспроизводимым и не требовал чрезмерного объема предварительной работы по поддержке SQL. Сводное сравнение рассмотренных тестов приведено в Таблице 1.

Тест	Данные	Сложность	Внедрение	Пригодность
ClickBench	Одна широкая таблица	Низкая: нет соединений	Низкие	Низкая
TPC-H	8 таблиц, близко к ЗНФ	Средняя: соединения, агрегации, подзапросы	Средние	Высокая
TPC-DS	Схема «снежинка», несколько таблиц фактов и измерений	Высокая: 99 шаблонов, широкий SQL	Высокие	Средняя
SSB	Схема «звезда», 5 таблиц	Низкая–средняя	Низкие; уже реализован	Низкая
JOB	IMDb, 21 таблица	Высокая только по соединениям	Средние	Средняя

Таблица 1: Сравнение эталонных тестов по критериям выбора

2.6. Выводы

SSB уже реализован в PosDB и подходит для базовой проверки колоночного движка, но его запросы слишком просты для поиска новых направлений развития. ClickBench также не является подходящим следующим шагом: он хорошо проверяет сканирование одной широкой таблицы, но не затрагивает соединения и подзапросы. JOB, наоборот, содержит много соединений и реальные данные, однако его основная цель — исследование качества оптимизатора, а не полноценное сравнение аналитического движка.

Основной выбор остается между TPC-H и TPC-DS. TPC-DS покрывает больше современных сценариев, но требует реализации 99 запросов и значительно более широкого подмножества SQL. Для текущего состояния PosDB это слишком высокий порог входа. TPC-H при этом содержит достаточно сложные запросы с соединениями, агрегациями и подзапросами, но остается существенно компактнее: 8 таблиц и 22 запроса.

Поэтому для дальнейшей реализации был выбран TPC-H. Он дает более сложную нагрузку, чем уже реализованный SSB, но при этом не требует настолько большого объема предварительной поддержки SQL, как TPC-DS.

3. Составление физических планов запросов

3.1. Основные принципы

Поскольку в PosDB на данный момент отсутствует оптимизатор, физические планы запросов из TPC-H составлялись вручную. Это позволяет сравнивать не качество оптимизатора, а работу самого движка при заданном плане выполнения. С другой стороны, такой подход требует явно описывать, почему для каждого запроса был выбран именно такой порядок операторов.

При составлении планов использовались следующие общие принципы.

Ранняя фильтрация [10]. Фильтры следует размещать как можно ближе к источникам данных. Это уменьшает количество позиций или кортежей, которые передаются в последующие операторы, и особенно важно перед соединениями с большими таблицами. В запросах TPC-H это чаще всего означает раннее применение предикатов по датам, регионам, типам деталей и другим селективным атрибутам.

Поздняя материализация [3]. В колоночной СУБД не всегда выгодно сразу собирать полноценные кортежи. До момента, когда значения действительно нужны для вычисления выражений или выдачи результата, между операторами можно передавать более легкое позиционное представление. Это уменьшает объем данных, читаемых из колонок и передаваемых между операторами. Особенно заметна польза поздней материализации в запросах, где итоговая выборка содержит широкие строковые атрибуты, но большая часть строк отбрасывается до финальной проекции.

Порядок соединений [5]. Порядок соединений влияет на размер промежуточных результатов. Обычно выгодно сначала соединять уже отфильтрованные таблицы или небольшие таблицы измерений, а самые крупные таблицы подключать после того, как для них можно построить более селективное условие. Для TPC-H это особенно важно из-за

таблицы `lineitem`, которая является самой крупной в наборе.

Полусоединения и антиполусоединения. В некоторых запросах не требуется переносить все значения из правой части соединения: достаточно проверить наличие или отсутствие подходящей строки. В таких случаях обычное соединение можно заменить на полусоединение или антиполусоединение. Это полезно для запросов с `EXISTS`, `IN`, `NOT EXISTS` и `NOT IN`, потому что такой оператор сохраняет только строки левой стороны и не размножает их при нескольких совпадениях справа.

Выбор физических операторов. Логический оператор может иметь разные физические реализации. Например, соединение может быть реализовано через хэш-соединение, соединение слиянием или соединение вложенными циклами. В составленных планах для соединения крупных таблиц в основном используется хэш-соединение, так как оно хорошо подходит для аналитических запросов без заранее заданного порядка строк.

3.2. Интересные случаи из составленных планов

Для всех запросов TPC-H были составлены физические планы, а для некоторых запросов несколько альтернативных вариантов. В этом разделе рассматриваются не все планы, а только несколько случаев, где выбор структуры плана особенно сильно влияет на объем промежуточных данных или показывает важное направление развития PosDB.

3.2.1. Q2: поздняя материализация после отбора минимальной цены

Запрос Q2 ищет поставщиков из Европы, которые предлагают детали заданного типа и размера по минимальной цене. Его сложность связана с тем, что условие на минимальную цену выражено через подзапрос:

```
ps_supplycost = (SELECT min(ps_supplycost) ...)
```

Прямой способ выполнения состоит в том, чтобы сначала вычислить минимальную цену для каждой детали, затем соединить этот результат с основной частью запроса и после этого отсортировать полученные строки. Такой план корректен, но у него есть недостаток: в промежуточный результат рано попадают широкие строковые поля поставщика, например `s_name`, `s_address`, `s_phone` и `s_comment`. Если большая часть кандидатов затем отбрасывается условием на минимальную цену, материализация этих колонок оказывается лишней.

Поэтому для Q2 был составлен вариант плана, в котором сначала вычисляются только узкие промежуточные результаты. Отдельная ветка строит пары

$$(\text{ps_partkey}, \min(\text{ps_supplycost}))$$

для подходящих деталей и европейских поставщиков. Вторая ветка строит набор кандидатов, но сохраняет только ключ детали, ключ поставщика и цену поставки. После соединения этих двух веток остаются только те пары “деталь–поставщик”, которые действительно имеют минимальную цену.

Только после этого выполняются соединения, необходимые для получения широких атрибутов результата: имени поставщика, адреса, телефона, комментария и производителя детали. Такой план хорошо иллюстрирует пользу поздней материализации: сначала выполняется отбор по компактным ключам и числовым значениям, а широкие строковые колонки читаются только для строк, которые пережили наиболее селективную часть запроса.

3.2.2. Q4: замена коррелированного EXISTS на полусоединение

Запрос Q4 считает количество заказов по приоритетам, но учитывает только те заказы, для которых существует хотя бы одна строка в `lineitem` с условием `l_commitdate < l_receiptdate`.

В исходной SQL-форме это выражается через коррелированный подзапрос с `EXISTS`. Прямое выполнение такого плана можно представить

как проверку подзапроса для каждого подходящего заказа: для каждой строки `orders` нужно найти хотя бы одну связанную строку `lineitem`. Такой вариант хорошо отражает исходную структуру запроса, но не является лучшим физическим планом.

Более эффективный вариант состоит в декорреляции подзапроса. Сначала фильтруется таблица `orders` по диапазону дат, затем отдельно фильтруется `lineitem` по условию `l_commitdate < l_receiptdate`. После этого выполняется полусоединение по ключам `o_orderkey = l_orderkey`.

Использование полусоединения здесь важно: запросу не нужны сами строки `lineitem` и не нужно учитывать их количество. Нужно только проверить, существует ли хотя бы одна подходящая строка. Поэтому обычное соединение могло бы размножить строки заказов, а полусоединение сохраняет семантику `EXISTS`: каждый заказ либо остается один раз, либо отбрасывается.

После полусоединения остается только выполнить агрегацию по `o_orderpriority` и отсортировать результат. Этот пример показывает, что для PosDB полезна поддержка явных операторов полусоединения и преобразований коррелированных подзапросов в соединения.

3.2.3. Q19: извлечение общих предикатов из дизъюнкции

Запрос Q19 содержит три большие ветви, соединенные через `OR`. Каждая ветвь описывает свой набор условий на `part` и `lineitem`: бренд, контейнер, размер детали, количество, способ доставки и инструкцию доставки. Если выполнить соединение `part` и `lineitem` сразу, а затем применить весь большой предикат целиком, то фильтрация сработает слишком поздно.

В составленном плане этот предикат разбивается на части, которые можно применить до соединения. Для таблицы `lineitem` выделяются условия, общие для всех трех ветвей:

```
l_shipmode IN ('AIR', 'AIR REG'),  
l_shipinstruct = 'DELIVER IN PERSON'.
```

Кроме того, из трех диапазонов по `l_quantity` выводится общий более широкий диапазон от 1 до 30. Он не заменяет исходное условие полностью, но позволяет отбросить строки, которые точно не подходят ни одной ветви. Аналогично для таблицы `part` строится предварительный фильтр как объединение условий на бренд, контейнер и размер детали из всех трех ветвей.

После этих предварительных фильтров выполняется соединение `part` и `lineitem`. Полный исходный предикат с тремя ветвями OR все равно применяется после соединения, чтобы сохранить точную семантику запроса. Таким образом, план использует безопасную предварительную фильтрацию: ранние предикаты могут пропустить лишние строки, но не могут удалить строки, которые должны попасть в результат.

Этот пример показывает, что для сложных дизъюнктивных условий полезно уметь выводить более слабые предикаты для отдельных таблиц. Такая техника особенно важна для колоночной СУБД, потому что уменьшает объем данных еще до соединения крупных таблиц.

3.3. Результаты

В результате были составлены физические планы для всех запросов ТРС-Н. Для части запросов были подготовлены несколько вариантов планов, чтобы проверить разные стратегии выполнения. Составленные планы не являются полноценной заменой оптимизатора. Их роль в данной работе другая: они задают достаточно эффективные варианты выполнения запросов, на которых можно исследовать производительность движка PosDB. В следующей части работы эти планы будут использованы для сравнения с PostgreSQL. При интерпретации результатов будет учитываться не только время выполнения, но и различия между физическими планами двух систем. Это позволит выделить случаи, где PosDB уже получает преимущество за счет колоночного хранения и позиционного представления, а также случаи, где для дальнейшего ускорения требуются новые операторы или оптимизации.

4. Реализация

Для проведения экспериментов потребовалось не только составить физические планы запросов TPC-H, но и реализовать в PosDB недостающую функциональность, необходимую для их запуска. Основной объем работы был связан с добавлением планов TPC-H, поддержкой отдельных SQL-конструкций и подготовкой инфраструктуры для сравнения с PostgreSQL.

Был добавлен отдельный файл с полным набором тестов TPC-H: `full.cpp`. В нем описаны физические планы для всех 22 запросов TPC-H. Для части запросов были реализованы несколько вариантов планов, чтобы сравнить разные стратегии выполнения. Всего в итоговом файле находится 36 тестовых сценариев, включая альтернативные планы. Размер файла с планами составляет около 6000 строк кода.

Для поддержки запросов TPC-H были реализованы или доработаны следующие элементы:

- оператор `LIMIT` для позиционного и кортежного представления;
- предикат `LIKE`;
- предикат `IN`;
- выражение `CASE`;
- выражение `SUBSTRING`;
- агрегат `COUNT(DISTINCT)`;
- агрегация с группировкой по выражениям.

Сводная информация об объеме изменений приведена в Таблице 2.

Таким образом, в рамках реализации был подготовлен полный набор физических планов TPC-H для PosDB и добавлена часть функциональности, без которой запуск этих запросов был невозможен. Эта работа также подготовила основу для экспериментальной части: теперь один и

Часть работы	Объем	Комментарий
Физические планы ТРС-Н	6056 строк	Итоговый размер тестов
Тестовые сценарии ТРС-Н	36 сценариев	22 основных запроса и альтернативные планы для части из них
Оператор LIMIT	6 новых файлов, около 265 добавленных строк	Реализация для позиционного и кортежного представления
Агрегация по выражениям	4 новых файла, около 551 добавленной строки	Нужна для запросов с вычисляемыми ключами группировки
Дополнительные выражения и предикаты	около 526 добавленных строк	LIKE, IN, CASE, SUBSTRING, COUNT(DISTINCT)

Таблица 2: Объем выполненной реализации

тот же запрос можно запускать в PosDB и PostgreSQL, проверять корректность результата и затем сравнивать не только время выполнения, но и используемые физические планы.

5. Эксперименты

В этой секции проводится сравнение производительности PosDB и PostgreSQL на запросах TPC-H. Цель эксперимента состоит не только в измерении времени выполнения, но и в интерпретации полученных результатов на основе физических планов. Для PosDB использовались составленные вручную планы запросов, а для PostgreSQL дополнительно анализировались планы, полученные с помощью EXPLAIN.

5.1. Условия эксперимента

Эксперименты проводились на данных TPC-H с коэффициентом масштабирования $SF=5$. Каждый запрос запускался 11 раз, после чего в качестве итогового значения выбиралась медиана времени выполнения. Перед каждым запуском очищался кеш, чтобы уменьшить влияние предыдущих запусков на результат. Для PostgreSQL было отключено параллельное выполнение запросов.

Это было сделано для того, чтобы сравнение отражало различия между движками и физическими планами, а не преимущество PostgreSQL за счет использования нескольких рабочих процессов. Планы PostgreSQL были получены отдельно с помощью EXPLAIN, время выполнения в таблице измерялось без EXPLAIN, так как его использование может вносить дополнительные накладные расходы.

Эксперименты проводились на следующей системе:

- модель устройства: HONOR NMH-WDX9;
- оперативная память: 16 ГиБ;
- процессор: AMD Ryzen 7 5700U with Radeon Graphics, 16 потоков;
- накопитель: SSD, 512 ГБ;
- операционная система: Ubuntu 24.04.4 LTS, 64-bit;
- версия ядра: Linux 6.17.0-20-generic.

5.2. Результаты

В Таблице 3 приведены результаты выполнения запросов TPC-H в PosDB и PostgreSQL. Последний столбец показывает отношение времени PostgreSQL к времени PosDB. Значение больше единицы означает, что PosDB выполнила запрос быстрее PostgreSQL. Если один из запросов выполнялся дольше одного часа, запуск останавливался, а в таблице указана соответствующая оценка.

Запрос	PosDB, мс	PostgreSQL, мс	PG / PosDB
Q1	15863	10000	0,63
Q2	2286	2030	0,89
Q3	3093	8540	2,76
Q4	2367	7180	3,03
Q5	11425	19310	1,69
Q6	1540	3500	2,27
Q7	3809	5260	1,38
Q8	20477	8050	0,39
Q9	87654	34750	0,40
Q10	5549	6070	1,09
Q11	5857	1010	0,17
Q12	2566	6450	2,51
Q13	36937	7040	0,19
Q14	3808	4430	1,16
Q15	> 1 ч	6600	< 0,002
Q16	4312	2720	0,63
Q17	56199	> 1 ч	> 64,06
Q18	38948	30590	0,79
Q19	2028	4450	2,19
Q20	6851	> 1 ч	> 525,47
Q21	> 1 ч	11020	< 0,004
Q22	> 1 ч	1810	< 0,001

Таблица 3: Сравнение времени выполнения запросов TPC-H

По итогам эксперимента PosDB оказалась быстрее PostgreSQL на 11 из 22 запросов. Наиболее заметное преимущество PosDB наблюдается на Q17 и Q20, где PostgreSQL не завершил выполнение за один час, а также на Q3, Q4, Q6, Q12 и Q19. PostgreSQL оказался существенно быстрее на Q11, Q13, Q15, Q21 и Q22. Эти результаты показывают, что PosDB уже получает преимущество на части аналитических запросов, но все еще сильно проигрывает в случаях, где требуется поддержка более сложных оптимизаций.

5.3. Интерпретация результатов

Q1. В Q1 PostgreSQL оказался быстрее: 10000 мс против 15863 мс у PosDB. Этот запрос сканирует почти всю таблицу `lineitem`, группирует строки по `l_returnflag` и `l_linestatus` и вычисляет сразу несколько агрегатов. Для колоночного хранения этот запрос не является идеальным случаем: используется много колонок одной таблицы, а фильтр по `l_shipdate` отбрасывает небольшую долю строк.

Основной проблемой, как мне кажется, является отсутствие возможности переиспользовать результаты некоторых выражений, а также в принципе не самая эффективная реализация дерева этих выражений. Таблица `lineitem` самая большая среди всех таблиц TPC-H, поэтому повторяющиеся вычисления и большое количество виртуальных вызовов при подсчете выражений приводят к сравнительно низкой производительности. Также, стоит заметить, что в фильтре использовался предикат $\geq$, истинной поддержки которого нет в PosDB, поэтому он реализуется через дизъюнкцию двух предикатов, что накладывает свой отпечаток при таком большом количестве строк.

Q2. В Q2 результаты близкие: 2286 мс у PosDB и 2030 мс у PostgreSQL. В плане PosDB запрос был разделен на ветку вычисления минимальной цены поставки и основную ветку. В первой ветке для каждой детали вычисляется `min(ps_supplycost)` среди европейских поставщиков, во второй ветке строятся кандидаты, после чего остается только проверить равенство стоимости минимальной. Такой план хорошо отражает структуру подзапроса и избегает коррелированного выполнения минимума для каждой строки.

Чтобы обойти PostgreSQL в данном запросе можно либо реализовать оператор **TopN**, либо реализовать оператор для сохранения промежуточных результатов, потому что большая часть работы дублируется в ветке с вычислением минимума и основной ветке.

Q3. Q3 является одним из удачных запросов для PosDB: 3093 мс против 8540 мс у PostgreSQL. В выбранном плане сначала фильтруются

`customer` по сегменту, `orders` по дате заказа и `lineitem` по дате отгрузки, затем выполняются хэш-соединения и агрегация по заказу. Такой план хорошо подходит PosDB: используется ограниченное число колонок, фильтры стоят до соединений, а итоговый результат после агрегации относительно небольшой.

Q4. В Q4 PosDB быстрее PostgreSQL примерно в три раза: 2367 мс против 7180 мс. Основная причина заключается в форме выполнения EXISTS. В плане PosDB коррелированный подзапрос переписан в хэш-полусоединение между отфильтрованными `orders` и `lineitem`. Это хорошо соответствует семантике запроса: нужно не получить строки из `lineitem`, а только проверить, существует ли хотя бы одна подходящая строка.

В плане PostgreSQL для этого запроса использовалось полусоединение с внутренним циклом с большим числом индексных обращений к `lineitem`. Такой план может быть хорош при небольшом числе строк слева, но здесь после фильтрации по дате остается много заказов.

Q5. В Q5 PosDB быстрее: 11425 мс против 19310 мс. Запрос содержит шесть таблиц, но в нем есть два сильных ограничения: регион ASIA и диапазон дат для заказов. В удачном плане сначала сокращаются маленькие таблицы `region` и `nation`, затем через них ограничиваются поставщики или клиенты, а `orders` фильтруется по 1994 году до соединения с `lineitem`.

При этом Q5 остается чувствительным к порядку соединений. Если слишком рано соединить `lineitem` с недостаточно отфильтрованной веткой, промежуточный результат становится слишком большим. В PosDB план здесь придерживается правила: сначала использовать селективные условия по региону и дате, а самую крупную таблицу `lineitem` подключать только после этого. Дополнительная проблема состоит в условии `c_nationkey = s_nationkey`, которое в текущем плане может применяться только как фильтр после соединения. Для таких случаев полезна поддержка соединений по составным ключам.

Q6. Q6 хорошо подходит для колоночной СУБД: PosDB выполняет его за 1540 мс, PostgreSQL — за 3500 мс. Запрос обращается только к таблице `lineitem`, применяет несколько фильтров по дате, скидке и количеству, а затем вычисляет одну сумму. В строковом хранении PostgreSQL при последовательном сканировании приходится обрабатывать строки таблицы целиком, тогда как PosDB достаточно прочитать только несколько нужных колонок.

Ранее не упоминалось, но в PosDB отсутствует специализированный тип данных для дат. В связи с этим фильтры в этом запросе были разбиты на несколько экземпляров для каждого отдельного условия и условия на дату были расположены позднее, чтобы сократить количество дорогостоящих сравнений строк, благодаря предварительно отфильтрованным данным по скидке и количеству.

Q7. В Q7 PosDB также быстрее: 3809 мс против 5260 мс. Важная часть плана — разбиение условия с `OR` на более простые фильтры по двум копиям таблицы `nation`. Вместо того чтобы применять полное условие только после всех соединений, PosDB заранее ограничивает обе стороны странами `FRANCE` и `GERMANY`. Полный предикат все равно применяется позже, чтобы сохранить точную семантику запроса, но раннее ограничение таблиц уменьшает объем промежуточных данных. Этот результат показывает пользу извлечения безопасных ранних предикатов из сложных условий.

Q8. В Q8 PostgreSQL значительно быстрее: 8050 мс против 20477 мс. Запрос содержит большое число соединений и вычисляет долю выручки по Бразилии относительно общей выручки за годы. Такой отрыв во времени выполнения возникает из-за удачного применения индексного сканирования в PostgreSQL. Сначала PostgreSQL соединяет все таблицы, которые имеют условия, способные сократить количество необходимых данных в `lineitem` (с 30 миллионов строк до приблизительно 500 тысяч), после чего получает необходимые кортежи `lineitem`. В PosDB нет возможности использовать такую технику, поэтому приходится ска-

нирывать `lineitem` целиком, сокращая количество данных с помощью `SemiJoin`, но полноценное сканирование всей крупной таблицы не позволяет быть быстрее.

Q9. Q9 оказался одним из самых тяжелых запросов для PosDB: 87654 мс против 34750 мс у PostgreSQL. Он содержит длинную цепочку соединений между `lineitem`, `orders`, `partsupp`, `part`, `supplier` и `nation`, а также фильтр по названию детали и группировку по стране и году. Здесь проигрыш, вероятно, заключается вновь в необходимости заменить индексное сканирование `lineitem` на `SemiJoin`, а также отсутствие соединения по составному условию, которое в итоге мы вынуждены представлять как соединение и последующий фильтр.

Q10. Q10 показывает почти равные результаты: PosDB выполняет запрос за 5549 мс, PostgreSQL за 6070 мс. Запрос содержит фильтр по дате заказа и условие на возвращенные товары, затем соединяет `customer`, `orders`, `lineitem` и `nation`. PosDB получает небольшой выигрыш за счет ранней фильтрации, чтения только нужных колонок и использования функциональной зависимости в агрегации. Однако преимущество невелико, как минимум за счет того, что в PostgreSQL используется оператор `TopN`, а в PosDB отсутствует истинная реализация предиката $\geq$.

Q11. В Q11 PostgreSQL существенно быстрее: 1010 мс против 5857 мс. Запрос работает с `partsupp`, `supplier` и `nation`, а также содержит подзапрос для вычисления порогового значения. Данные в этом запросе сравнительно небольшие по сравнению с запросами, активно использующими `lineitem`, поэтому преимущество колоночного хранения почти не проявляется. PostgreSQL эффективно выполняет небольшие соединения и агрегации. PosDB использует практически аналогичный план, в котором обеим СУБД приходится дублировать часть работы. Поэтому, здесь ситуация, вероятно, схожа с Q1, где проблема заключалась в менее эффективном дереве выражений. Дополнительно, для PosDB здесь

было бы полезно материализовать общий агрегированный результат и использовать его повторно.

Q12. Q12 является удачным для PosDB: 2566 мс против 6450 мс. Запрос соединяет `orders` и `lineitem`, фильтрует строки по датам и способу доставки, а затем считает два условных агрегата. Читается небольшое количество атрибутов, а фильтры очень селективны, поэтому запрос особенно удачен для колоночного исполнения. PostgreSQL также может выполнить этот запрос последовательным сканированием и хэш-соединением, но PosDB выигрывает за счет меньшего объема читаемых данных и раннего применения фильтров к `lineitem`.

Q13. Q13 является неудачным для PosDB: 36937 мс против 7040 мс у PostgreSQL. Запрос требует посчитать распределение клиентов по количеству заказов, исключая заказы с определенным комментарием. Обычно такой запрос выражается через внешнее соединение `customer` и `orders`, затем группировку по клиенту и повторную группировку по числу заказов. В PosDB нет необходимого левого соединения, из-за чего приходится дублировать значительную часть операторов. Без реализации этого оператора, вряд ли возможно превзойти PostgreSQL по производительности.

Q14. В итоговой таблице Q14 немного быстрее в PosDB: 3808 мс против 4430 мс. Запрос вычисляет долю выручки от промо-товаров за заданный месяц и соединяет `lineitem` с `part`. Если использовать план, идентичный плану PostgreSQL, то PosDB проигрывает на этом запросе. Возможно, причиной этому не самая эффективная реализация функций агрегации. План, который достиг наилучшего результата использовал полусоединение, чтобы избежать соединения `lineitem` с `part`, но это привело к тому, что для вычисления финального агрегата приходится сканировать `lineitem` дважды.

Q15. Q15 не завершился в PosDB за один час, тогда как PostgreSQL выполнил его за 6600 мс. Запрос сначала вычисляет выручку поставщиков за период, а затем выбирает поставщика с максимальной выручкой. Для эффективного выполнения важно один раз построить агрегированный результат и переиспользовать его при поиске максимума. В текущем плане PosDB такой сценарий выполнить невозможно, поэтому для улучшения результата нужно реализовать функциональность для переиспользования общих промежуточных результатов.

Q16. Для Q16 итоговый вариант PosDB выполняется за 4312 мс, PostgreSQL за 2720 мс. Запрос исключает поставщиков с определенным комментарием и считает число уникальных поставщиков по группам деталей. Среди всех вариантов плана наилучшим образом в PosDB показал план с использованием антиполусоединения, но так и не смог обогнать PostgreSQL. Предположительно, PostgreSQL быстрее засчет индексного сканирования и более эффективной реализации COUNT(DISTINCT).

Q17. Q17 является самым сильным выигрышем PosDB: 56199 мс против более одного часа у PostgreSQL. Запрос содержит коррелированный подзапрос, в котором количество товара сравнивается с долей от среднего количества для той же детали. В плане PosDB этот подзапрос был преобразован в предварительную агрегацию `lineitem` по ключу детали с последующим соединением с основной частью запроса. Такая декорреляция устраняет повторное вычисление среднего значения для каждой строки. PostgreSQL построил существенно менее удачный план, в котором не удалось декоррелировать подзапрос.

Q18. Q18 в итоговой таблице выполняется медленнее в PosDB: 38948 мс против 30590 мс у PostgreSQL. Запрос сначала находит крупные заказы по суммарному количеству товаров, а затем соединяет их с `customer`, `orders` и `lineitem` для получения итоговой информации. Здесь не удалось составить оптимальный план исполнения: либо прихо-

дится сканировать `lineitem` дважды, чтобы получить нужный агрегат для подзапроса и итогового результата, либо же попытаться использовать его повторно (потому что это одинаковый агрегат), но тогда агрегация выполняется в самом начале плана, что приводит к невозможности поздней материализации. Если бы существовал оператор для переиспользования существующих результатов, запрос в PosDB удалось бы значительно ускорить.

Q19. В Q19 PosDB удалось превзойти PostgreSQL: 2028 мс против 4450 мс. Ключевая причина — вынос части условий из большого дизъюнктивного предиката. Исходный запрос содержит три ветви через `OR`, каждая из которых задает условия на `part` и `lineitem`. В плане PosDB общие и безопасные условия были применены до соединения. Полный исходный предикат проверяется после соединения, но ранние фильтры уже существенно уменьшают объем данных.

Q20. Q20 также сильно лучше в PosDB: 6851 мс против более одного часа у PostgreSQL. Запрос содержит коррелированный подзапрос, который PostgreSQL не смог декоррелировать. В то время как в PosDB удалось составить удачный план с использованием `SemiJoin`.

Q21. Q21 не завершился в PosDB за один час, тогда как PostgreSQL выполнил его за 11020 мс. Это один из самых сложных запросов TPC-H с точки зрения логики: он содержит несколько условий существования и отсутствия строк в `lineitem`, фактически требуя самосоединений, полусоединений и антиполусоединений. PostgreSQL строит план с комбинацией хэш-соединений, `Nested Loop Semi Join` и `Nested Loop Anti Join`, используя индексные обращения к `lineitem`. В PosDB также удалось декоррелировать этот запрос, однако на текущей системе не хватает оперативной памяти для его исполнения из-за многократного сканирования `lineitem`. Индексное сканирование смогло бы решить эту проблему, оно же и является основным преимуществом PostgreSQL в этом запросе. Варианты же планов, где используются зависимые опе-

раторы, выполняются слишком долго.

Q22. Q22 также оказался неудачным для PosDB: он не завершился за один час, тогда как PostgreSQL выполнил его за 1810 мс. Запрос фильтрует клиентов по префиксу телефона, сравнивает баланс со средним значением по выбранным клиентам и оставляет только клиентов без заказов. Для данного запроса не удалось составить эффективный план с текущим набором операторов. При подсчете среднего значения используется оператор агрегации, который принудительно материализует результат, однако текущая реализация `AntiSemiJoin` работает исключительно с позиционными операторами. Это приводит к неудачному расположению фильтра и необходимости лишнего сканирования достаточно крупной таблицы

Заключение

Целью работы являлось исследование производительности движка PosDB на запросах из эталонного теста для аналитических СУБД и сравнение полученных результатов с PostgreSQL, при этом для каждого запроса требовалось сопоставить физический план PosDB с планом PostgreSQL и объяснить причины различий в производительности.

Для достижения цели были выполнены следующие задачи:

1. Описаны ключевые характеристики и преимущества некоторых известных эталонных тестов, для реализации выбран эталонный тест TPC-H.
2. Реализована недостающая функциональность PosDB, необходимая для выполнения запросов TPC-H.
3. Вручную составлены физические планы для всех 22 запросов TPC-H, включая несколько альтернативных вариантов.
4. Проведено сравнение времени выполнения запросов в PosDB и PostgreSQL, результаты сопоставлены с физическими планами обеих систем.
5. На основе анализа определены особенности PosDB, дающие преимущество, и отсутствующие операторы и оптимизации, ограничивающие производительность.

Список литературы

- [1] Abadi Daniel J., Boncz Peter A., Harizopoulos Stavros. Column-oriented database systems // *Proc. VLDB Endow.* — 2009. — Vol. 2, no. 2. — P. 1664–1665. — URL: <https://doi.org/10.14778/1687553.1687625>.
- [2] Boncz Peter, Neumann Thomas, Erling Orri. *TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark* // Revised Selected Papers of the 5th TPC Technology Conference on Performance Characterization and Benchmarking - Volume 8391. — Berlin, Heidelberg : Springer-Verlag, 2013. — P. 61–76. — URL: https://doi.org/10.1007/978-3-319-04936-6_5.
- [3] A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store / George A. Chernishev, Viacheslav Galaktionov, Valentin V. Grigorev et al. // Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP) co-located with the 25th International Conference on Extending Database Technology and the 25th International Conference on Database Theory (EDBT/ICDT 2022), Edinburgh, UK, March 29, 2022 / Ed. by Kostas Stefanidis, Lukasz Golab. — Vol. 3130 of CEUR Workshop Proceedings. — CEUR-WS.org, 2022. — P. 21–30. — URL: <http://ceur-ws.org/Vol-3130/paper3.pdf>.
- [4] How Good Are Query Optimizers, Really? / Viktor Leis, Andrey Gubichev, Atanas Mirchev et al. // Proceedings of the VLDB Endowment. — 2015. — Vol. 9, no. 3. — P. 204–215. — URL: <https://www.vldb.org/pvldb/vol9/p204-leis.pdf>.
- [5] Ilyas Ihab F., Aref Walid G., Elmagarmid Ahmed K. Supporting top-K join queries in relational databases // Proceedings of the 29th International Conference on Very Large Data Bases - Volume 29. — VLDB '03. — VLDB Endowment, 2003. — P. 754–765.

- [6] O’Neil P E, O’Neil E J, Chen X. The Star Schema Benchmark (SSB). — 2009.
- [7] Poess Meikel, Rabl Tilmann, Jacobsen Hans-Arno. [Analysis of TPC-DS: the first standard benchmark for SQL-based big data systems](#) // Proceedings of the 2017 Symposium on Cloud Computing. — SoCC ’17. — New York, NY, USA : Association for Computing Machinery, 2017. — P. 573–585. — URL: <https://doi.org/10.1145/3127479.3128603>.
- [8] PosDB: An Architecture Overview / George A. Chernishev, Vyacheslav Galaktionov, Valentin D. Grigorev et al. // [Program. Comput. Softw.](#) — 2018. — Vol. 44, no. 1. — P. 62–74. — URL: <https://doi.org/10.1134/S0361768818010024>.
- [9] A Study of PosDB Performance in a Distributed Environment / George Chernishev, Viacheslav Galaktionov, Valentin Grigorev et al. // Proceedings of the Second Conference on Software Engineering and Information Management (SEIM-2017) (full papers). — Vol. 1864 of CEUR Workshop Proceedings. — Saint-Petersburg, Russia : CEUR-WS.org, 2017. — URL: <http://ceur-ws.org/Vol-1864/>.
- [10] Yan Cong, Lin Yin, He Yeye. Predicate Pushdown for Data Science Pipelines // [Proc. ACM Manag. Data.](#) — 2023. — Vol. 1, no. 2. — 28 p. — URL: <https://doi.org/10.1145/3589281>.